

UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE COMPUTAÇÃO - FACOM

CAUÊ FERNANDO FARIA ABREU
IGOR DOS REIS MASCARENHAS
JÚLIA YASMIN SILVA GUIMARÃES
LEANDRO EVARISTO DE SOUSA
MATEUS ALVES VIEGAS
SANMYLA SILVA COSTA

Relatório Técnico - Etapa 2
TEMA 2 — Syscall getppid() e
Escalonador FCFS (First-Come, First-Served)

Trabalho apresentado como relatório técnico para a obtenção de nota na disciplina de Sistemas Operacionais.

Docente: Prof. Marcela Zanchetta.

1. INTRODUÇÃO

Este relatório técnico documenta a Etapa 2 do projeto da disciplina de Sistemas Operacionais, cujo objetivo é integrar conceitos teóricos e práticos por meio da modificação e experimentação no sistema operacional xv6, com QEMU. Contudo, a Etapa 2 é um complemento da Etapa 1 que já foi entregue.

O tema escolhido foi o Tema 2, que consiste em realizar:

- Chamada de sistema `getppid()`: permite que um processo consulte o PID do seu processo pai, possibilitando identificar a hierarquia de processos do sistema.
- Escalonador FCFS (First-Come, First-Served): substitui o escalonador Round Robin original do xv6 por uma política não preemptiva, em que o processo que chegou primeiro à fila de prontos é executado até o fim, antes de qualquer outro.

Para ter acesso ao trabalho completo acesse o link: [Repositório aqui](#)

2. ARQUIVOS MODIFICADOS

2.1. kernel/syscall.h

Adicionados os números de chamada de sistema e `SYS_getppid` (24).

```
#define SYS_getppid 24
```

2.2. kernel/syscall.c

Adicionados os protótipos extern de `sys_getppid()`, e suas entradas no vetor `syscalls[]` que mapeia número da syscall para a função correspondente.

```
extern uint64 sys_getppid(void);  
[SYS_getppid] sys_getppid,
```

2.3. kernel/sysproc.c

Implementada a função `sys_getppid()`, que retorna o PID do processo pai (`p->parent->pid`) ou 1 caso o processo não tenha pai.

```
uint64  
sys_getppid(void) {  
    struct proc *p = myproc();  
    if (p->parent != 0) {  
        return p->parent->pid;  
    } return 1; }  

```

2.4. user/user.h

Adicionados os protótipos `int getppid(void)`; para uso nos programas de espaço de usuário.

```
int getppid(void);
```

2.5. user/[usys.pl](#)

Adicionadas as chamadas `entry("getppid")`, que geram os stubs em assembly (`usys.S`) responsáveis pelo `ecall` para o kernel.

```
entry("getppid");
```

2.6. kernel/proc.c

Reescrita a função `scheduler()`: em vez de round robin, percorre a tabela de processos e escolhe o processo `RUNNABLE` de menor PID (critério de ordem de chegada / FCFS). A função `yield()` foi neutralizada (`no-op`) para remover a preempção por tempo.

```
// Encontrar o processo RUNNABLE com o menor PID (o que chegou primeiro)
for (p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if (p->state == RUNNABLE) {
        // Se for o primeiro que encontramos, ou se tiver um PID menor
        // que o selecionado
        if (first_proc == 0 || p->pid < first_proc->pid) {
            if (first_proc != 0) {
                release(&first_proc->lock); // Libera o anterior escolhido
            }
            first_proc = p; // Guarda o novo candidato mais antigo
            continue;       // Mantém o lock de 'first_proc' retido
        }
    }
    release(&p->lock);
}
```

// A função completa está no repositório do Git

2.7. user/teste.c

```
#define NUM_FILHOS 4
for (i = 0; i < NUM_FILHOS; i++) {
    int child_pid = fork();
    if (child_pid < 0) {
        printf("erro no fork\n");
        exit(1);
    }
}
```

Continuação no Git: [Repositório aqui](#)

3. RESULTADOS

3.1. FCFS

PID	Chegada	Início	Término	Response Time	Turnaround	Burst
4	31	31	34	0	3	3
5	31	34	36	3	5	2
6	31	36	41	5	10	5
7	31	42	43	11	12	1
Médias:				4,75	7,5	2,75

3.2. ROUND ROBIN(RR)

PID	Chegada	Início	Término	Response Time	Turnaround	Burst
4	38	39	44	1	6	5
5	40	41	50	1	10	9
6	40	42	52	2	12	10
7	40	43	51	3	11	8
Médias:				1,75	9,75	8

3.3. DADOS REUNIDOS

Métrica média (ticks)	FCFS	Round Robin
Response Time médio	4,75	1,75
Turnaround médio	7,5	9,75
Burst médio (CPU)	2,75	8
Tempo total	12	14

3.4. GRÁFICO

FCFS e Round Robin

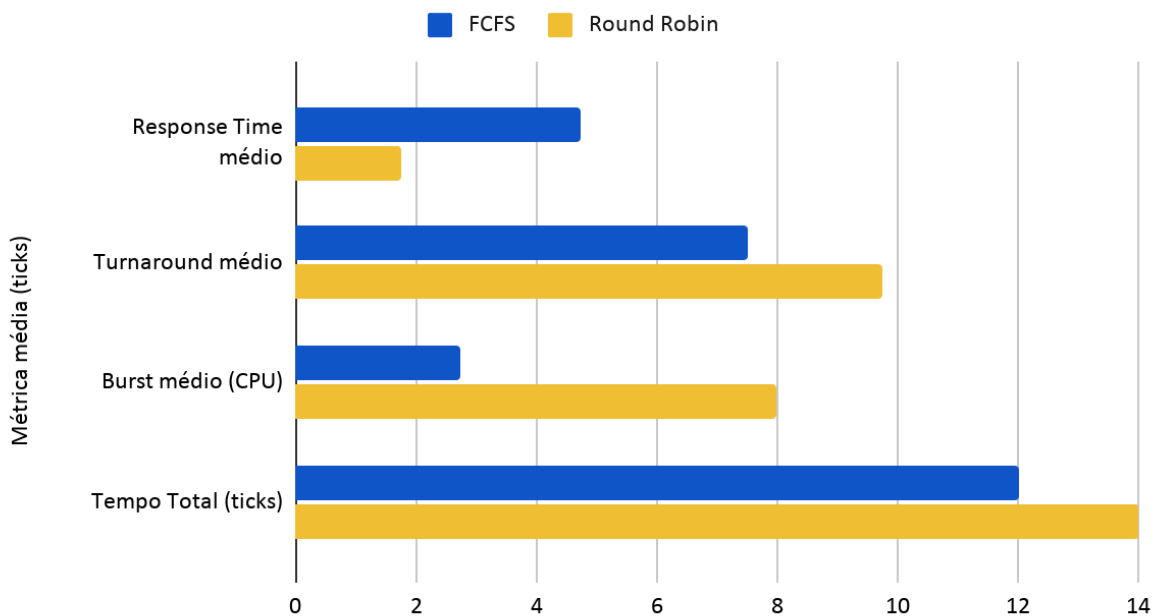


Figura 1 - Gráfico das médias métricas

4. ANÁLISE

A implementação da função `sys_getppid()` acessa o campo `p->parent` sem utilizar o `wait_lock`. De acordo com a própria convenção do xv6, esse bloqueio deveria proteger esse acesso contra possíveis alterações concorrentes, como o reparentamento de processos durante sua finalização. Apesar disso, esse comportamento não comprometeu os testes realizados.

Outro ponto observado é que o critério utilizado no FCFS, baseado no menor PID, considera que os processos permanecem no estado **RUNNABLE** desde sua criação até o momento da execução. Caso um processo seja bloqueado (por operações de entrada e saída ou chamadas como sleep) e depois volte para a fila de prontos, ele continuará tendo prioridade pelo PID, e não pela ordem em que retornou à fila. Essa simplificação foi suficiente para os testes realizados, que utilizaram apenas processos **CPU-bound**, mas difere da implementação clássica do FCFS em cenários com operações de I/O.

Os resultados obtidos confirmam o comportamento esperado de cada algoritmo de escalonamento. No **FCFS**, o **Response Time** aumentou progressivamente entre os processos (0, 3, 5 e 11 ticks), evidenciando o chamado **efeito comboio (convoy effect)**. Como o algoritmo não possui preempção, cada processo precisa esperar que todos os anteriores terminem sua execução. Assim, o último processo da fila (PID 7) apresentou o maior tempo de espera, mesmo sendo o processo com menor tempo de execução (burst de 1 tick).

No **Round Robin**, o comportamento foi diferente. O **Response Time** permaneceu baixo e mais equilibrado entre todos os processos (1, 1, 2 e 3 ticks), mostrando que o algoritmo distribui melhor o tempo de CPU entre os processos prontos. Em compensação, o **Turnaround Time** médio foi um pouco maior (9,75 ticks contra 7,5 ticks do FCFS), já que cada processo é interrompido e retomado várias vezes até finalizar sua execução.

O **tempo total de execução** foi bastante semelhante nos dois algoritmos: **12 ticks no FCFS** e **14 ticks no Round Robin**. Isso mostra que, para uma carga composta apenas por processos que utilizam CPU, o algoritmo de escalonamento não altera significativamente o tempo necessário para concluir todo o conjunto de processos. O que muda é a forma como esse tempo é distribuído entre eles.

Vale destacar também um pequeno intervalo ocioso de 1 tick observado entre o término do PID 6 (tick 41) e o início do PID 7 (tick 42) na execução FCFS. Mesmo o algoritmo sendo não preemptivo, o escalonador só reavalia a fila de processos prontos na próxima iteração do loop principal, o que introduz esse atraso mínimo entre a conclusão de um processo e o início do próximo, mesmo quando o processo seguinte já está disponível para execução.

Por fim, o *Waiting Time* não foi calculado neste trabalho, pois isso exigiria instrumentar outras partes do kernel para contabilizar todos os períodos de espera dos processos ao longo das preempções. Como essa implementação ficou fora do escopo do projeto, essa métrica não foi considerada nas análises.

5. CONCLUSÕES

O desenvolvimento deste projeto permitiu colocar em prática diversos conceitos estudados na disciplina de Sistemas Operacionais.

A implementação da chamada de sistema **getppid()** exigiu compreender como ocorre a comunicação entre o espaço do usuário e o kernel no xv6, desde o registro da nova syscall até a geração da instrução `ecall` por meio do `usys.pl`.

Além disso, a substituição do escalonador **Round Robin** por uma política **FCFS** proporcionou uma compreensão mais aprofundada do funcionamento interno do escalonador do xv6, incluindo os estados dos processos (**RUNNABLE** e **RUNNING**), bem como o funcionamento das funções `sched()` e `swtch()`. Também foi possível observar como uma pequena alteração no critério de escolha do próximo processo pode modificar significativamente o comportamento do sistema.

Os experimentos realizados confirmaram, na prática, as características conhecidas de cada algoritmo. O **FCFS** é simples de implementar e possui menor custo com trocas de contexto, porém sofre com o efeito comboio, aumentando o tempo de espera dos processos que chegam depois. Já o **Round Robin** oferece uma resposta mais rápida e equilibrada para todos os processos, mas aumenta um pouco o tempo total de conclusão devido às interrupções e retomadas frequentes. Dessa forma, nenhum algoritmo pode ser considerado superior em todas as situações; a escolha depende dos objetivos e das características do sistema.

Durante o desenvolvimento também foi identificado um problema importante no ambiente de testes. Inicialmente, o QEMU estava configurado para utilizar **três núcleos de CPU (CPUS = 3)**, permitindo que processos fossem executados em paralelo. Isso causava inconsistências nos resultados e nos logs devido às escritas simultâneas no console. Após alterar a configuração para **CPUS = 1**, foi possível realizar experimentos em um ambiente de processador único, obtendo resultados consistentes e compatíveis com o comportamento esperado dos algoritmos de escalonamento. Essa experiência reforçou a importância de validar o ambiente experimental antes de analisar e interpretar os resultados obtidos.

6. REFERÊNCIAS

PDOS/MIT. xv6: a simple, Unix-like teaching operating system. Disponível em: <https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf>.

Documentação oficial do xv6 (6.1810, MIT). Disponível em: <https://pdos.csail.mit.edu/6.1810/2024/xv6.html>.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Operating System Concepts**. 10. ed. Hoboken: John Wiley & Sons, 2018.

Slides de Aula